

Cricbuzz LiveStats · 25 SQL queries

Tested against a copy of the database · run one at a time in the Neon SQL Editor (neondb)

How to check: run each query, compare the row count and first row with the yellow line under it. If both match, the query is done. Lines starting with -- are notes explaining the choices; copy them too, they become the explanations on the SQL Analytics page.

Beginner · Questions 1–8

Q1 Players who represent India

```
-- id: 1
-- title: Players who represent India
-- question: Find all players who represent India. Display their full name, playing role, batting style, and bowling style.
-- params: none

SELECT  player_name,
        gender,
        playing_role,
        batting_style,
        bowling_style
FROM    dim_player
WHERE   primary_team = 'India'
ORDER BY gender DESC, player_name;
```

Expected: 209 rows · **First row:** A Kumble | male | Bowler | |

Q2 Matches in the last 30 days

```
-- id: 2
-- title: Matches in the last 30 days
-- question: Show all matches played in the last 30 days with description, both teams, venue with city and date. Most recent first.
-- params: none

-- "Last 30 days" counts back from the latest match in the archive, not from
-- today, so the answer does not change depending on the day it is run.
SELECT  t1.team_display || ' vs ' || t2.team_display
        || ', ' || s.series_name          AS match_description,
        t1.team_display                  AS team_1,
        t2.team_display                  AS team_2,
        v.venue_name || ', ' || v.city    AS venue,
        m.match_date
FROM    fact_match m
JOIN    dim_team   t1 ON t1.team_id = m.team1_id
JOIN    dim_team   t2 ON t2.team_id = m.team2_id
JOIN    dim_venue  v  ON v.venue_id = m.venue_id
JOIN    dim_series s  ON s.series_id = m.series_id
WHERE   m.match_date >= (SELECT MAX(match_date) FROM fact_match) - 30
ORDER BY m.match_date DESC;
```

Expected: 11 rows · **First row:** South Africa vs Namibia, South Africa tour of Namibia | South Africa | Namibia | Namibia Cricket Ground, Windhoek | 2026-09-09

Q3 Top 10 ODI run scorers

```
-- id: 3
-- title: Top 10 ODI run scorers
-- question: List the top 10 highest run scorers in ODI cricket with total runs, batting average and number of centuries.
-- params: none

SELECT  p.player_name,
        p.gender,
        SUM(b.runs_scored)                                AS total_runs,
        ROUND(SUM(b.runs_scored)::numeric
              / NULLIF(COUNT(*) FILTER (WHERE NOT b.is_not_out), 0), 2)
              AS batting_average,
        COUNT(*) FILTER (WHERE b.runs_scored >= 100)      AS centuries
FROM    fact_batting b
JOIN    dim_player   p ON p.player_id = b.player_id
JOIN    fact_match    m ON m.match_id = b.match_id
WHERE   m.match_format = 'ODI'
GROUP BY p.player_id, p.player_name, p.gender
ORDER BY total_runs DESC
LIMIT 10;
```

Expected: 10 rows · **First row:** V Kohli | male | 14819 | 58.34 | 54

Q4 Venues with capacity over 50,000

```
-- id: 4
-- title: Venues with capacity over 50,000
-- question: Display all venues with a seating capacity of more than 50,000. Show venue name, city, country and capacity. Largest first.
-- params: none

-- Capacity is today's figure, not the capacity on the day of each match.
-- 29 small grounds have no published capacity and are left out by the filter.
SELECT  venue_name,
        city,
        country,
        capacity
FROM    dim_venue
WHERE   capacity > 50000
ORDER BY capacity DESC;
```

Expected: 7 rows · **First row:** Narendra Modi Stadium | Ahmedabad | India | 132000

Q5 Wins by team

```
-- id: 5
-- title: Wins by team
-- question: Calculate how many matches each team has won. Show team name and total wins, most wins first.
-- params: none

-- Matches with no result or a tie have no winner (winner_id is NULL),
-- so the JOIN leaves them out automatically.
SELECT  t.team_display AS team,
        COUNT(*)      AS total_wins
FROM    fact_match m
JOIN    dim_team   t ON t.team_id = m.winner_id
GROUP BY t.team_id, t.team_display
ORDER BY total_wins DESC, team;
```

Expected: 43 rows · **First row:** India | 313

Q6 Players by playing role

```
-- id: 6
-- title: Players by playing role
-- question: Count how many players belong to each playing role.
-- params: none

-- Roles are worked out from what players actually did (how much they
-- bowled, where they batted, how many catches and stumpings), because the
-- source does not record an official role. "Unknown" = too few matches.
SELECT  playing_role,
        COUNT(*) AS players
FROM    dim_player
GROUP BY playing_role
ORDER BY players DESC;
```

Expected: 5 rows · **First row:** Bowler | 1224

Q7 Highest individual score by format

```
-- id: 7
-- title: Highest individual score by format
-- question: Find the highest individual batting score in each cricket format (Test, ODI, T20I).
-- params: none

-- The archive holds ODIs only, so one row comes back. The query itself is
-- written for any number of formats.
SELECT  m.match_format      AS format,
        MAX(b.runs_scored)  AS highest_score
FROM    fact_batting b
JOIN    fact_match  m ON m.match_id = b.match_id
GROUP BY m.match_format
ORDER BY m.match_format;
```

Expected: 1 rows · **First row:** ODI | 264

Q8 Series that started in 2024

```
-- id: 8
-- title: Series that started in 2024
-- question: Show all series that started in 2024 with series name, host country, match type, start date and total matches.
-- params: none

-- Host country = the country (or countries) where the series' matches were
-- played. "Matches planned" is not in the source; this counts the matches
-- actually played. series_id 0 is the "no series" placeholder and is skipped.
SELECT  s.series_name,
        STRING_AGG(DISTINCT v.country, ', ') AS host_country,
        MIN(m.match_format)                  AS match_type,
        s.gender,
        s.first_match                         AS start_date,
        COUNT(DISTINCT m.match_id)           AS total_matches
FROM    dim_series s
JOIN    fact_match m ON m.series_id = s.series_id
JOIN    dim_venue  v ON v.venue_id = m.venue_id
WHERE   s.series_id <> 0
        AND EXTRACT(YEAR FROM s.first_match) = 2024
GROUP BY s.series_id, s.series_name, s.gender, s.first_match
ORDER BY s.first_match, s.series_name;
```

Expected: 39 rows · **First row:** Zimbabwe tour of Sri Lanka | Sri Lanka | ODI | male | 2024-01-06 | 3

Intermediate · Questions 9–16

Q9 All-rounders with 1000 runs and 50 wickets

```
-- id: 9
-- title: All-rounders with 1000 runs and 50 wickets
-- question: Find all-rounders with more than 1000 runs AND more than 50 wickets. Show player name, total
runs, total wickets and format.
-- params: none

-- Runs and wickets are added up in two SEPARATE steps (the two CTEs) and
-- only then joined. Joining batting and bowling rows first and summing
-- afterwards would multiply every total – a classic SQL trap.
WITH runs AS (
    SELECT  b.player_id, m.match_format, SUM(b.runs_scored) AS total_runs
    FROM    fact_batting b
    JOIN    fact_match  m ON m.match_id = b.match_id
    GROUP BY b.player_id, m.match_format
),
wickets AS (
    SELECT  w.player_id, m.match_format, SUM(w.wickets) AS total_wickets
    FROM    fact_bowling w
    JOIN    fact_match  m ON m.match_id = w.match_id
    GROUP BY w.player_id, m.match_format
)
SELECT  p.player_name,
        p.primary_team,
        r.total_runs,
        w.total_wickets,
        r.match_format AS format
FROM    runs      r
JOIN    wickets   w ON w.player_id   = r.player_id
                AND w.match_format = r.match_format
JOIN    dim_player p ON p.player_id   = r.player_id
WHERE   p.playing_role = 'All-rounder'
        AND r.total_runs  > 1000
        AND w.total_wickets > 50
ORDER BY r.total_runs DESC;
```

Expected: 59 rows · **First row:** TM Dilshan | Sri Lanka | 9212 | 87 | ODI

Q10 Last 20 completed matches

```
-- id: 10
-- title: Last 20 completed matches
-- question: Details of the last 20 completed matches: description, both teams, winner, victory margin,
victory type and venue. Most recent first.
-- params: none

-- "Completed" = the match produced a winner. No-results and ties are skipped.
SELECT  t1.team_display || ' vs ' || t2.team_display
        || ', ' || s.series_name                AS match_description,
        t1.team_display                        AS team_1,
        t2.team_display                        AS team_2,
        w.team_display                        AS winner,
        m.victory_margin,
        m.victory_type,
        v.venue_name || ', ' || v.city          AS venue,
        m.match_date
FROM    fact_match m
JOIN    dim_team  t1 ON t1.team_id = m.team1_id
JOIN    dim_team  t2 ON t2.team_id = m.team2_id
JOIN    dim_team  w  ON w.team_id  = m.winner_id
JOIN    dim_venue v  ON v.venue_id = m.venue_id
JOIN    dim_series s  ON s.series_id = m.series_id
WHERE   m.winner_id IS NOT NULL
ORDER BY m.match_date DESC, m.match_id DESC
LIMIT 20;
```

Expected: 20 rows · **First row:** South Africa vs Namibia, South Africa tour of Namibia | South Africa | Namibia | South Africa | 99 | runs | Namibia Cricket Ground, Windhoek

Q11 Runs across formats

```
-- id: 11
-- title: Runs across formats
-- question: For players who have played at least 2 formats, show total runs in Test, ODI and T20I, plus
overall batting average.
-- params: none

-- The archive holds ODIs only, so no player has 2 formats and this returns
-- 0 rows. That is the correct answer for this data. Delete the HAVING line
-- to see every player's ODI column filled in.
SELECT  p.player_name,
        SUM(b.runs_scored) FILTER (WHERE m.match_format = 'Test') AS test_runs,
        SUM(b.runs_scored) FILTER (WHERE m.match_format = 'ODI') AS odi_runs,
        SUM(b.runs_scored) FILTER (WHERE m.match_format = 'T20I') AS t20i_runs,
        ROUND(SUM(b.runs_scored)::numeric
              / NULLIF(COUNT(*) FILTER (WHERE NOT b.is_not_out), 0), 2)
          AS overall_average
FROM    fact_batting b
JOIN    fact_match   m ON m.match_id = b.match_id
JOIN    dim_player   p ON p.player_id = b.player_id
GROUP BY p.player_id, p.player_name
HAVING  COUNT(DISTINCT m.match_format) >= 2
ORDER BY SUM(b.runs_scored) DESC;
```

Expected: 0 rows · **First row:** (no rows – expected, see comment)

Q12 Home vs away wins

```
-- id: 12
-- title: Home vs away wins
-- question: For each team, count wins at home and away, where home means the venue's country matches the
team's country.
-- params: none

-- Step 1 turns each match into TWO rows, one per team, so every team can
-- be judged on its own. Step 2 labels the venue from that team's point of
-- view: its own ground = Home, the opponent's ground = Away, anywhere
-- else = Neutral. home_nation is used rather than country so that West
-- Indies are at home in Jamaica, Barbados and Trinidad.
WITH team_match AS (
    SELECT match_id, team1_id AS team_id, team2_id AS opponent_id, venue_id, winner_id FROM fact_match
    UNION ALL
    SELECT match_id, team2_id, team1_id, venue_id, winner_id FROM fact_match
),
labelled AS (
    SELECT  t.team_display,
           CASE WHEN v.home_nation = t.team_name THEN 'Home'
                WHEN v.home_nation = o.team_name THEN 'Away'
                ELSE 'Neutral' END AS venue_type,
           (tm.winner_id = tm.team_id) AS won
    FROM    team_match tm
    JOIN    dim_team  t ON t.team_id = tm.team_id
    JOIN    dim_team  o ON o.team_id = tm.opponent_id
    JOIN    dim_venue v ON v.venue_id = tm.venue_id
)
SELECT  team_display AS team,
        COUNT(*) FILTER (WHERE venue_type = 'Home') AS home_matches,
        COUNT(*) FILTER (WHERE venue_type = 'Home' AND won) AS home_wins,
        COUNT(*) FILTER (WHERE venue_type = 'Away') AS away_matches,
        COUNT(*) FILTER (WHERE venue_type = 'Away' AND won) AS away_wins,
        COUNT(*) FILTER (WHERE venue_type = 'Neutral') AS neutral_matches,
        COUNT(*) FILTER (WHERE venue_type = 'Neutral' AND won) AS neutral_wins
FROM    labelled
GROUP BY team_display
ORDER BY COUNT(*) FILTER (WHERE won) DESC, team;
```

Expected: 44 rows · **First row:** India | 191 | 126 | 232 | 119 | 103 | 68

Q13 100-run partnerships between consecutive batters

```
-- id: 13
-- title: 100-run partnerships between consecutive batters
-- question: Find partnerships where two consecutive batsmen (batting positions next to each other) put on
100 or more runs in the same innings.
-- params: none

-- fact_partnership holds real stands (the runs added while the two were
-- at the crease together). fact_batting is joined twice, once per batter,
-- to fetch their batting positions and keep only neighbours (differ by 1).
SELECT  p1.player_name          AS batter_1,
        p2.player_name          AS batter_2,
        pt.runs                  AS partnership_runs,
        pt.innings_no,
        pt.wicket_no             AS for_wicket,
        tb.team_display          AS batting_team,
        m.match_date
FROM    fact_partnership pt
JOIN    fact_batting b1 ON b1.match_id = pt.match_id AND b1.innings_no = pt.innings_no
        AND b1.player_id = pt.batter1_id
JOIN    fact_batting b2 ON b2.match_id = pt.match_id AND b2.innings_no = pt.innings_no
        AND b2.player_id = pt.batter2_id
JOIN    dim_player  p1 ON p1.player_id = pt.batter1_id
JOIN    dim_player  p2 ON p2.player_id = pt.batter2_id
JOIN    dim_team    tb ON tb.team_id   = pt.batting_team_id
JOIN    fact_match   m  ON m.match_id   = pt.match_id
WHERE   ABS(b1.batting_position - b2.batting_position) = 1
        AND pt.runs >= 100
ORDER BY pt.runs DESC, m.match_date DESC;
```

Expected: 1,422 rows · **First row:** MN Samuels | CH Gayle | 372 | 1 | 2 | West Indies | 2015-02-24

Q14 Bowling performance by venue

```
-- id: 14
-- title: Bowling performance by venue
-- question: For bowlers with at least 3 matches at the same venue (bowling at least 4 overs in each), show
average economy, total wickets and matches at that venue.
-- params: none

-- 4 overs = 24 legal balls. Spells shorter than that are removed FIRST
-- (WHERE), then the 3-match rule is applied to what is left (HAVING).
SELECT  p.player_name          AS bowler,
        v.venue_name || ', ' || v.city AS venue,
        COUNT(DISTINCT w.match_id)    AS matches,
        ROUND(AVG(w.runs_conceded * 6.0 / w.balls_bowled), 2) AS avg_economy,
        SUM(w.wickets)                AS total_wickets
FROM    fact_bowling w
JOIN    fact_match   m ON m.match_id = w.match_id
JOIN    dim_venue    v ON v.venue_id = m.venue_id
JOIN    dim_player    p ON p.player_id = w.player_id
WHERE   w.balls_bowled >= 24
GROUP BY p.player_id, p.player_name, v.venue_id, v.venue_name, v.city
HAVING  COUNT(DISTINCT w.match_id) >= 3
ORDER BY total_wickets DESC, avg_economy;
```

Expected: 3,407 rows · **First row:** Shakib Al Hasan | Shere Bangla National Stadium, Dhaka | 74 | 4.26 | 110

Q15 Performance in close matches

```
-- id: 15
-- title: Performance in close matches
-- question: In close matches (won by under 50 runs or under 5 wickets), show each player's average runs,
close matches played and close matches won by their team.
-- params: none

-- Only players with 10+ close-match innings are listed, so one lucky knock
-- doesn't top the table.
WITH close_matches AS (
    SELECT match_id, winner_id
    FROM fact_match
    WHERE (victory_type = 'runs' AND victory_margin < 50)
        OR (victory_type = 'wickets' AND victory_margin < 5)
)
SELECT p.player_name,
       p.primary_team,
       COUNT(DISTINCT b.match_id) AS close_matches,
       ROUND(AVG(b.runs_scored), 2) AS avg_runs,
       COUNT(DISTINCT b.match_id) FILTER (WHERE b.team_id = c.winner_id) AS close_matches_won
FROM fact_batting b
JOIN close_matches c ON c.match_id = b.match_id
JOIN dim_player p ON p.player_id = b.player_id
GROUP BY p.player_id, p.player_name, p.primary_team
HAVING COUNT(DISTINCT b.match_id) >= 10
ORDER BY avg_runs DESC;
```

Expected: 636 rows · **First row:** UT Khawaja | Australia | 16 | 58.81 | 11

Q16 Batting by year since 2020

```
-- id: 16
-- title: Batting by year since 2020
-- question: For matches since 2020, show each player's average runs per match and average strike rate per
year, for players with 5+ matches that year.
-- params: none

SELECT p.player_name,
       d.year,
       COUNT(DISTINCT b.match_id) AS matches,
       ROUND(AVG(b.runs_scored), 2) AS avg_runs,
       ROUND(AVG(b.runs_scored * 100.0 / NULLIF(b.balls_faced, 0)), 2) AS avg_strike_rate
FROM fact_batting b
JOIN fact_match m ON m.match_id = b.match_id
JOIN dim_date d ON d.date_key = m.date_key
JOIN dim_player p ON p.player_id = b.player_id
WHERE d.year >= 2020
GROUP BY p.player_id, p.player_name, d.year
HAVING COUNT(DISTINCT b.match_id) >= 5
ORDER BY p.player_name, d.year;
```

Expected: 1,636 rows · **First row:** A Athanaze | 2023 | 7 | 34.29 | 87.95

Advanced · Questions 17–25

Q17 Does winning the toss help?

```
-- id: 17
-- title: Does winning the toss help?
-- question: What percentage of matches are won by the toss winner, broken down by toss decision (bat first or bowl first)?
-- params: none

-- Matches with no winner (no result, abandoned, tie) are excluded: they
-- can't be won by anyone, so they would drag every percentage down.
SELECT CASE toss_decision WHEN 'bat' THEN 'Chose to bat first'
      WHEN 'field' THEN 'Chose to bowl first' END AS toss_decision,
      COUNT(*) AS matches,
      COUNT(*) FILTER (WHERE toss_winner_id = winner_id) AS toss_winner_won,
      ROUND(100.0 * COUNT(*) FILTER (WHERE toss_winner_id = winner_id)
        / COUNT(*), 2) AS win_pct
FROM fact_match
WHERE winner_id IS NOT NULL
      AND toss_decision IS NOT NULL
GROUP BY toss_decision
ORDER BY toss_decision;
```

Expected: 2 rows · **First row:** Chose to bat first | 1484 | 706 | 47.57

Q18 Most economical bowlers

```
-- id: 18
-- title: Most economical bowlers
-- question: Most economical bowlers in limited-overs cricket (ODI and T20): overall economy and total wickets, for bowlers with 10+ matches averaging 2+ overs per match.
-- params: none

-- Economy is total runs ÷ total overs, not the average of each match's
-- economy – a 1-over spell must not count as much as a 10-over one.
SELECT p.player_name AS bowler,
      p.primary_team,
      COUNT(DISTINCT w.match_id) AS matches,
      ROUND(SUM(w.balls_bowled) / 6.0, 1) AS overs,
      ROUND(SUM(w.runs_conceded) * 6.0 / NULLIF(SUM(w.balls_bowled), 0), 2) AS economy,
      SUM(w.wickets) AS total_wickets
FROM fact_bowling w
JOIN fact_match m ON m.match_id = w.match_id
JOIN dim_player p ON p.player_id = w.player_id
WHERE m.match_format IN ('ODI', 'T20I', 'T20')
GROUP BY p.player_id, p.player_name, p.primary_team
HAVING COUNT(DISTINCT w.match_id) >= 10
      AND SUM(w.balls_bowled) / 6.0 / COUNT(DISTINCT w.match_id) >= 2
ORDER BY economy, total_wickets DESC;
```

Expected: 872 rows · **First row:** P Siaka | Papua New Guinea | 11 | 82.2 | 2.95 | 18

Q19 Most consistent batters since 2022

```
-- id: 19
-- title: Most consistent batters since 2022
-- question: Average runs and standard deviation of runs per batter since 2022, counting only innings of 10+
balls. Lower deviation = more consistent.
-- params: none

-- Two extra rules, both needed for a sensible answer:
-- * 10+ qualifying innings – a deviation over 2 or 3 innings means nothing.
-- * average of 30+ – otherwise tail-enders who ALWAYS score little top the
list as the "most consistent batters".
SELECT p.player_name,
       p.primary_team,
       COUNT(*)                AS innings,
       ROUND(AVG(b.runs_scored), 2) AS avg_runs,
       ROUND(STDDEV(b.runs_scored), 2) AS std_dev_runs
FROM   fact_batting b
JOIN   fact_match   m ON m.match_id = b.match_id
JOIN   dim_player   p ON p.player_id = b.player_id
WHERE  m.match_date >= DATE '2022-01-01'
      AND b.balls_faced >= 10
GROUP BY p.player_id, p.player_name, p.primary_team
HAVING  COUNT(*) >= 10
      AND AVG(b.runs_scored) >= 30
ORDER BY std_dev_runs;
```

Expected: 211 rows · **First row:** M Levitt | Netherlands | 13 | 34.00 | 12.91

Q20 Matches and averages by format

```
-- id: 20
-- title: Matches and averages by format
-- question: For players with 20+ matches, show the number of Test, ODI and T20 matches and the batting
average in each.
-- params: none

-- Test and T20 columns are 0 / empty because the archive holds ODIs only.
SELECT p.player_name,
       COUNT(DISTINCT b.match_id) FILTER (WHERE m.match_format = 'Test')                AS test_matches,
       COUNT(DISTINCT b.match_id) FILTER (WHERE m.match_format = 'ODI')                AS odi_matches,
       COUNT(DISTINCT b.match_id) FILTER (WHERE m.match_format IN ('T20I', 'T20')) AS t20_matches,
       ROUND(SUM(b.runs_scored) FILTER (WHERE m.match_format = 'Test')::numeric
             / NULLIF(COUNT(*) FILTER (WHERE m.match_format = 'Test' AND NOT b.is_not_out), 0), 2) AS
test_avg,
       ROUND(SUM(b.runs_scored) FILTER (WHERE m.match_format = 'ODI')::numeric
             / NULLIF(COUNT(*) FILTER (WHERE m.match_format = 'ODI' AND NOT b.is_not_out), 0), 2) AS
odi_avg,
       ROUND(SUM(b.runs_scored) FILTER (WHERE m.match_format IN ('T20I', 'T20'))::numeric
             / NULLIF(COUNT(*) FILTER (WHERE m.match_format IN ('T20I', 'T20') AND NOT b.is_not_out), 0), 2)
AS t20_avg
FROM   fact_batting b
JOIN   fact_match   m ON m.match_id = b.match_id
JOIN   dim_player   p ON p.player_id = b.player_id
GROUP BY p.player_id, p.player_name
HAVING  COUNT(DISTINCT b.match_id) >= 20
ORDER BY odi_matches DESC;
```

Expected: 792 rows · **First row:** V Kohli | 0 | 300 | 0 | | 58.34 |

Q21 Overall player ranking

```
-- id: 21
-- title: Overall player ranking
-- question: Rank players in each format with one weighted score combining batting (runs, average, strike
rate), bowling (wickets, average, economy) and fielding (catches, stumpings).
-- params: none

-- Formula from the brief:
--   batting = runs × 0.01 + batting_avg × 0.5 + strike_rate × 0.3
--   bowling = wickets × 2 + (50 - bowling_avg) × 0.5 + (6 - economy) × 2
--   fielding = catches × 3 + stumpings × 5
-- Bowling points apply only to players who have taken a wicket (a bowling
-- average needs at least one). Minimum 20 matches so the ranking isn't led
-- by someone with one great game. Top 25 per format.
WITH appearances AS (
    SELECT match_id, player_id FROM fact_batting
    UNION
    SELECT match_id, player_id FROM fact_bowling
),
matches AS (
    SELECT a.player_id, m.match_format, COUNT(*) AS matches
    FROM appearances a
    JOIN fact_match m ON m.match_id = a.match_id
    GROUP BY a.player_id, m.match_format
),
batting AS (
    SELECT b.player_id, m.match_format,
        SUM(b.runs_scored) AS runs,
        SUM(b.runs_scored)::numeric
        / NULLIF(COUNT(*) FILTER (WHERE NOT b.is_not_out), 0) AS bat_avg,
        SUM(b.runs_scored) * 100.0 / NULLIF(SUM(b.balls_faced), 0) AS strike_rate
    FROM fact_batting b
    JOIN fact_match m ON m.match_id = b.match_id
    GROUP BY b.player_id, m.match_format
),
bowling AS (
    SELECT w.player_id, m.match_format,
        SUM(w.wickets) AS wickets,
        SUM(w.runs_conceded)::numeric / NULLIF(SUM(w.wickets), 0) AS bowl_avg,
        SUM(w.runs_conceded) * 6.0 / NULLIF(SUM(w.balls_bowled), 0) AS economy
    FROM fact_bowling w
    JOIN fact_match m ON m.match_id = w.match_id
    GROUP BY w.player_id, m.match_format
),
fielding AS (
    -- A catch is credited to the fielder; in "caught and bowled" the bowler
    -- is the catcher.
    SELECT CASE WHEN b.dismissal_kind = 'caught and bowled'
        THEN b.dismissed_by_id ELSE b.fielder_id END AS player_id,
        m.match_format,
        COUNT(*) FILTER (WHERE b.dismissal_kind IN ('caught', 'caught and bowled')) AS catches,
        COUNT(*) FILTER (WHERE b.dismissal_kind = 'stumped') AS stumpings
    FROM fact_batting b
    JOIN fact_match m ON m.match_id = b.match_id
    WHERE b.dismissal_kind IN ('caught', 'caught and bowled', 'stumped')
    GROUP BY 1, m.match_format
),
scored AS (
    SELECT mt.player_id, mt.match_format, mt.matches,
        COALESCE(bt.runs, 0) AS runs,
        COALESCE(bw.wickets, 0) AS wickets,
        COALESCE(f.catches, 0) AS catches,
        COALESCE(f.stumpings, 0) AS stumpings,
        COALESCE(bt.runs * 0.01 + COALESCE(bt.bat_avg, 0) * 0.5
        + COALESCE(bt.strike_rate, 0) * 0.3, 0) AS batting_pts,
        CASE WHEN bw.wickets > 0
            THEN bw.wickets * 2 + (50 - bw.bowl_avg) * 0.5 + (6 - bw.economy) * 2
            ELSE 0 END AS bowling_pts,
        COALESCE(f.catches, 0) * 3 + COALESCE(f.stumpings, 0) * 5 AS fielding_pts
    FROM matches mt
    LEFT JOIN batting bt ON bt.player_id = mt.player_id AND bt.match_format = mt.match_format
    LEFT JOIN bowling bw ON bw.player_id = mt.player_id AND bw.match_format = mt.match_format
    LEFT JOIN fielding f ON f.player_id = mt.player_id AND f.match_format = mt.match_format
```

```

WHERE mt.matches >= 20
),
ranked AS (
  SELECT s.*,
         batting_pts + bowling_pts + fielding_pts AS total_score,
         RANK() OVER (PARTITION BY match_format
                     ORDER BY batting_pts + bowling_pts + fielding_pts DESC) AS format_rank
  FROM   scored s
)
SELECT r.match_format AS format,
       r.format_rank AS rank,
       p.player_name,
       p.playing_role,
       r.matches, r.runs, r.wickets, r.catches, r.stumpings,
       ROUND(r.batting_pts, 1) AS batting_pts,
       ROUND(r.bowling_pts, 1) AS bowling_pts,
       ROUND(r.fielding_pts, 1) AS fielding_pts,
       ROUND(r.total_score, 1) AS total_score
FROM   ranked r
JOIN   dim_player p ON p.player_id = r.player_id
WHERE  r.format_rank <= 25
ORDER BY r.match_format, r.format_rank;

```

Expected: 25 rows . **First row:** ODI | 1 | MS Dhoni | Wicket-keeper | 281 | 10274 | 1 | 308 | 118
 | 154.5 | 13.2 | 1514.0 | 1681.7

Q22 Head-to-head records (last 3 years)

```
-- id: 22
-- title: Head-to-head records (last 3 years)
-- question: For team pairs with 5+ matches in the last 3 years: matches, wins each, average victory margin,
wins batting first vs chasing, venues used, and win % for each team.
-- params: none

-- Each pair is stored once, lower team_id first (team_a), so India–Australia
-- and Australia–India count as the same rivalry. "Last 3 years" counts back
-- from the latest match in the archive. Margins are split into runs and
-- wickets, because averaging "40 runs" with "6 wickets" means nothing.
WITH recent AS (
    SELECT m.*,
           LEAST(m.team1_id, m.team2_id) AS team_a,
           GREATEST(m.team1_id, m.team2_id) AS team_b,
           i.batting_team_id AS batted_first
    FROM fact_match m
    JOIN fact_innings i ON i.match_id = m.match_id AND i.innings_no = 1
    WHERE m.match_date >= (SELECT MAX(match_date) FROM fact_match) - INTERVAL '3 years'
)
SELECT ta.team_display AS team_a,
       tb.team_display AS team_b,
       COUNT(*) AS matches,
       COUNT(DISTINCT r.venue_id) AS venues,
       COUNT(*) FILTER (WHERE winner_id = team_a) AS team_a_wins,
       COUNT(*) FILTER (WHERE winner_id = team_b) AS team_b_wins,
       ROUND(100.0 * COUNT(*) FILTER (WHERE winner_id = team_a) / COUNT(*), 1) AS team_a_win_pct,
       ROUND(100.0 * COUNT(*) FILTER (WHERE winner_id = team_b) / COUNT(*), 1) AS team_b_win_pct,
       ROUND(AVG(victory_margin) FILTER (WHERE winner_id = team_a AND victory_type = 'runs'), 1) AS
a_avg_win_runs,
       ROUND(AVG(victory_margin) FILTER (WHERE winner_id = team_a AND victory_type = 'wickets'), 1) AS
a_avg_win_wkts,
       ROUND(AVG(victory_margin) FILTER (WHERE winner_id = team_b AND victory_type = 'runs'), 1) AS
b_avg_win_runs,
       ROUND(AVG(victory_margin) FILTER (WHERE winner_id = team_b AND victory_type = 'wickets'), 1) AS
b_avg_win_wkts,
       COUNT(*) FILTER (WHERE winner_id = team_a AND batted_first = team_a) AS a_wins_batting_first,
       COUNT(*) FILTER (WHERE winner_id = team_a AND batted_first <> team_a) AS a_wins_chasing,
       COUNT(*) FILTER (WHERE winner_id = team_b AND batted_first = team_b) AS b_wins_batting_first,
       COUNT(*) FILTER (WHERE winner_id = team_b AND batted_first <> team_b) AS b_wins_chasing
FROM recent r
JOIN dim_team ta ON ta.team_id = r.team_a
JOIN dim_team tb ON tb.team_id = r.team_b
GROUP BY ta.team_display, tb.team_display
HAVING COUNT(*) >= 5
ORDER BY matches DESC, team_a;
```

Expected: 45 rows · **First row:** Australia Women | India Women | 14 | 8 | 12 | 2 | 85.7 | 14.3 | 104.3 | 5.5 | 102.0 | 5.0 | 6 | 6 | 1 | 1

Q23 Recent form

```
-- id: 23
-- title: Recent form
-- question: Using each player's last 10 innings: average of last 5 vs last 10, strike-rate trend, scores of
50+, consistency (std dev), and a form label.
-- params: none

-- ROW_NUMBER numbers each player's innings newest-first (1 = latest).
-- Only players who have batted within the last year of the archive and
-- have 10 innings are rated, so retired players don't get a "form".
-- Form rules (average of the last 5 innings):
--   Excellent  45+ and at least two 50s in the last 10
--   Good       30+
--   Average    18+
--   Poor       below 18
WITH numbered AS (
    SELECT  b.player_id, b.runs_scored, b.balls_faced, m.match_date,
            ROW_NUMBER() OVER (PARTITION BY b.player_id
                                ORDER BY m.match_date DESC, m.match_id DESC) AS rn
    FROM    fact_batting b
    JOIN    fact_match   m ON m.match_id = b.match_id
),
last10 AS (
    SELECT  player_id,
            MAX(match_date)                                AS last_innings,
            COUNT(*)                                        AS innings,
            AVG(runs_scored) FILTER (WHERE rn <= 5)        AS avg_last5,
            AVG(runs_scored)                                AS avg_last10,
            SUM(runs_scored) FILTER (WHERE rn <= 5) * 100.0
              / NULLIF(SUM(balls_faced) FILTER (WHERE rn <= 5), 0) AS sr_last5,
            SUM(runs_scored) * 100.0 / NULLIF(SUM(balls_faced), 0) AS sr_last10,
            COUNT(*) FILTER (WHERE runs_scored >= 50)      AS fifties,
            STDDEV(runs_scored)                            AS std_dev
    FROM    numbered
    WHERE   rn <= 10
    GROUP BY player_id
)
SELECT  p.player_name,
        p.primary_team,
        l.last_innings,
        ROUND(l.avg_last5, 1)                AS avg_last5,
        ROUND(l.avg_last10, 1)               AS avg_last10,
        ROUND(l.sr_last5, 1)                 AS sr_last5,
        ROUND(l.sr_last10, 1)                AS sr_last10,
        CASE WHEN l.sr_last5 > l.sr_last10 THEN 'Rising'
              WHEN l.sr_last5 < l.sr_last10 THEN 'Falling'
              ELSE 'Flat' END                 AS sr_trend,
        l.fifties                             AS fifties_last10,
        ROUND(l.std_dev, 1)                   AS consistency_std_dev,
        CASE WHEN l.avg_last5 >= 45 AND l.fifties >= 2 THEN 'Excellent Form'
              WHEN l.avg_last5 >= 30                      THEN 'Good Form'
              WHEN l.avg_last5 >= 18                      THEN 'Average Form'
              ELSE 'Poor Form' END              AS form
FROM    last10 l
JOIN    dim_player p ON p.player_id = l.player_id
WHERE   l.innings = 10
        AND l.last_innings >= (SELECT MAX(match_date) FROM fact_match) - INTERVAL '1 year'
ORDER BY l.avg_last5 DESC;
```

Expected: 371 rows · **First row:** JE Root | England | 2026-07-19 | 87.0 | 62.5 | 95.6 | 91.4 | Rising | 7 | 39.8 | Excellent Form

Q24 Best batting partnerships

```
-- id: 24
-- title: Best batting partnerships
-- question: For pairs of consecutive batsmen (positions differ by 1) with 5+ partnerships: average runs,
stands over 50, highest stand and success rate. Rank the best pairs.
-- params: none

-- LEAST/GREATEST put each pair in a fixed order, so "Rohit & Dhawan" and
-- "Dhawan & Rohit" are counted as one pair. Success rate = share of their
-- stands that reached 50 (a "good" partnership).
WITH stands AS (
    SELECT  LEAST(pt.batter1_id, pt.batter2_id) AS player_a,
            GREATEST(pt.batter1_id, pt.batter2_id) AS player_b,
            pt.runs
    FROM    fact_partnership pt
    JOIN    fact_batting b1 ON b1.match_id = pt.match_id AND b1.innings_no = pt.innings_no
            AND b1.player_id = pt.batter1_id
    JOIN    fact_batting b2 ON b2.match_id = pt.match_id AND b2.innings_no = pt.innings_no
            AND b2.player_id = pt.batter2_id
    WHERE   ABS(b1.batting_position - b2.batting_position) = 1
)
SELECT  RANK() OVER (ORDER BY AVG(s.runs) DESC) AS rank,
        pa.player_name || ' & ' || pb.player_name AS pair,
        COUNT(*) AS partnerships,
        ROUND(AVG(s.runs), 1) AS avg_runs,
        COUNT(*) FILTER (WHERE s.runs >= 50) AS fifty_plus_stands,
        MAX(s.runs) AS highest,
        ROUND(100.0 * COUNT(*) FILTER (WHERE s.runs >= 50) / COUNT(*), 1) AS success_rate_pct
FROM    stands s
JOIN    dim_player pa ON pa.player_id = s.player_a
JOIN    dim_player pb ON pb.player_id = s.player_b
GROUP BY s.player_a, s.player_b, pa.player_name, pb.player_name
HAVING COUNT(*) >= 5
ORDER BY rank;
```

Expected: 1,454 rows · **First row:** 1 | RG Sharma & KL Rahul | 10 | 98.5 | 6 | 227 | 60.0

Q25 Career trajectory by quarter

```
-- id: 25
-- title: Career trajectory by quarter
-- question: Track each player's quarterly runs and strike rate, compare each quarter with the previous one,
and label the career phase (Ascending, Declining, Stable). Players with 6+ quarters of 3+ matches only.
-- params: none

-- Step 1 quarterly: one row per player per quarter (3+ matches only).
-- Step 2 compared: LAG() fetches the previous quarter so each quarter can
-- be labelled Improving / Declining / Stable (±10% on average runs).
-- Step 3 per player: count the labels and compare the first 3 quarters
-- with the last 3. Last 3 more than 10% higher = Career Ascending,
-- more than 10% lower = Career Declining, otherwise Career Stable.
WITH quarterly AS (
    SELECT  b.player_id,
            d.year,
            d.quarter,
            d.year || '-Q' || d.quarter AS period,
            COUNT(DISTINCT b.match_id) AS matches,
            AVG(b.runs_scored) AS avg_runs,
            SUM(b.runs_scored) * 100.0 / NULLIF(SUM(b.balls_faced), 0) AS strike_rate
    FROM    fact_batting b
    JOIN    fact_match m ON m.match_id = b.match_id
    JOIN    dim_date d ON d.date_key = m.date_key
    GROUP BY b.player_id, d.year, d.quarter
    HAVING  COUNT(DISTINCT b.match_id) >= 3
),
compared AS (
    SELECT  q.*,
            LAG(avg_runs) OVER w AS prev_avg,
            ROW_NUMBER() OVER w AS q_first,
            ROW_NUMBER() OVER (PARTITION BY player_id
                                ORDER BY year DESC, quarter DESC) AS q_last,
            COUNT(*) OVER (PARTITION BY player_id) AS quarters
    FROM    quarterly q
    WINDOW w AS (PARTITION BY player_id ORDER BY year, quarter)
),
summary AS (
    SELECT  player_id,
            MAX(quarters) AS quarters,
            MIN(period) AS first_quarter,
            MAX(period) AS last_quarter,
            COUNT(*) FILTER (WHERE avg_runs > prev_avg * 1.1) AS improving_q,
            COUNT(*) FILTER (WHERE avg_runs < prev_avg * 0.9) AS declining_q,
            COUNT(*) FILTER (WHERE prev_avg IS NOT NULL
                                AND avg_runs BETWEEN prev_avg * 0.9 AND prev_avg * 1.1) AS stable_q,
            AVG(avg_runs) FILTER (WHERE q_first <= 3) AS early_avg,
            AVG(avg_runs) FILTER (WHERE q_last <= 3) AS recent_avg,
            AVG(strike_rate) FILTER (WHERE q_first <= 3) AS early_sr,
            AVG(strike_rate) FILTER (WHERE q_last <= 3) AS recent_sr
    FROM    compared
    GROUP BY player_id
    HAVING  MAX(quarters) >= 6
)
SELECT  p.player_name,
        p.primary_team,
        s.quarters,
        s.first_quarter,
        s.last_quarter,
        ROUND(s.early_avg, 1) AS early_avg_runs,
        ROUND(s.recent_avg, 1) AS recent_avg_runs,
        ROUND(s.early_sr, 1) AS early_strike_rate,
        ROUND(s.recent_sr, 1) AS recent_strike_rate,
        s.improving_q,
        s.declining_q,
        s.stable_q,
        CASE WHEN s.recent_avg > s.early_avg * 1.1 THEN 'Career Ascending'
              WHEN s.recent_avg < s.early_avg * 0.9 THEN 'Career Declining'
              ELSE 'Career Stable' END AS career_phase
FROM    summary s
JOIN    dim_player p ON p.player_id = s.player_id
ORDER BY s.recent_avg - s.early_avg DESC;
```

Expected: 526 rows · **First row:** AJ Healy | Australia | 20 | 2014-Q4 | 2026-Q1 | 12.4 | 51.0 | 96.5 | 118.4 | 10 | 6 | 3 | Career Ascending